

Coworking Space Service Extension

Arquitetura de Microsserviços, CI/CD na AWS e Orquestração com EKS

Engenharia DevOps | Documentação Hands-on & Guia Didático

October 2, 2026

Resumo Executivo

Este documento apresenta o ciclo completo de vida operacional do microsserviço **Coworking Space Analytics Service**. A solução abrange desde o provisionamento declarativo do cluster gerenciado **Amazon EKS**, containerização com **Docker**, pipeline de Integração Contínua (CI) automatizada via **AWS CodeBuild** e armazenamento no **AWS ECR**, até a implantação orientada a microsserviços via **Kubernetes** integrado a um banco de dados relacional **PostgreSQL** provisionado por **Helm**.

1 A História das Etapas: Crônica da Jornada Operacional

A implementação foi dividida em marcos cronológicos, simulando o desafio de transformar uma base de código monolítica em um microsserviço escalável em nuvem.

- Etapa 1 – Conexão e Prontidão do Ambiente Local:** Inicialmente, a máquina de desenvolvimento precisou ser instrumentada com o conjunto de ferramentas DevOps: instalação do **AWS CLI v2**, configuração das credenciais temporárias do IAM via STS (*Security Token Service*), e instalação do **eksctl**, **kubectl** e **helm**.
- Etapa 2 – Provisionamento da Infraestrutura Kubernetes (EKS):** Utilizando o utilitário **eksctl**, foi disparada a criação automatizada de uma Virtual Private Cloud (VPC) dedicada, subnets públicas/privadas, control plane gerenciado e um *managed node group* baseado em instâncias EC2 (**t3.small**). A comunicação foi validada com a emissão do **kubeconfig**.
- Etapa 3 – Implantação e Carga do Banco PostgreSQL via Helm:** Para evitar a escrita manual e repetitiva de manifestos complexos, utilizou-se o gerenciador de pacotes **Helm** com o repositório oficial da Bitnami. Após a inicialização do pod **postgresql-0**, foram executados os scripts DDL e de carga (*seed*) das tabelas de usuários e tokens de acesso corporativo.
- Etapa 4 – Containerização e Resolução de Conflitos Python:** Construiu-se um **Dockerfile** otimizado baseado em **python:3.10-slim**. Durante o ciclo de testes, foi diagnosticado e sanado um erro crítico de retrocompatibilidade do ecossistema Python (incompatibilidade entre o *Flask 2.2.2* e o *Werkzeug 3.x*), congelando-se a dependência estritamente em **Werkzeug==2.2.2**.
- Etapa 5 – Pipeline de Integração Contínua (AWS CodeBuild & ECR):** Criou-se um repositório no **Amazon ECR** denominado **coworking-analytics**. Em seguida, definiu-se a pipeline declarativa **buildspec.yml** integrada ao repositório GitHub. O AWS CodeBuild autenticou no registro de imagens, compilou o contêiner e aplicou versionamento semântico (**1.0.0** e **latest**).
- Etapa 6 – Orquestração Kubernetes e Ajuste Fino de Probes:** A aplicação foi implantada no cluster com manifestos nativos (**ConfigMap**, **Deployment** e **Service**). Identificou-se um gargalo no ciclo de vida de inicialização, onde o **readinessProbe** original apontava para um modelo não

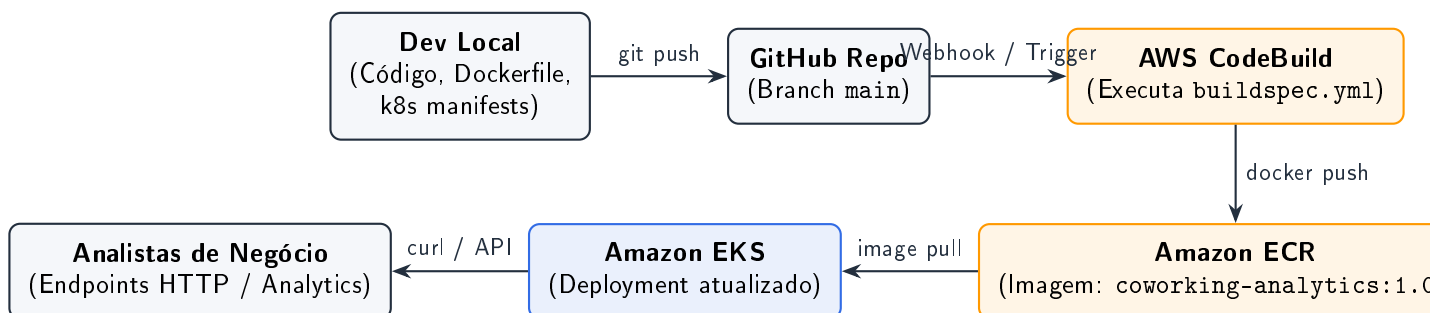
instanciado. O probe foi redirecionado para a rota `/health_check`, garantindo a estabilização do pod em estado **1/1 Running**.

7. **Etapa 7 – Validação dos Endpoints & Observabilidade:** Por meio de túnel seguro (*port-forwarding*), as rotas de analítica corporativa (`/api/reports/daily_usage` e `/api/reports/user_visits`) foram testadas com sucesso total, consumindo os registros populados no PostgreSQL.

2 Fluxogramas de Arquitetura e Pipeline (TikZ)

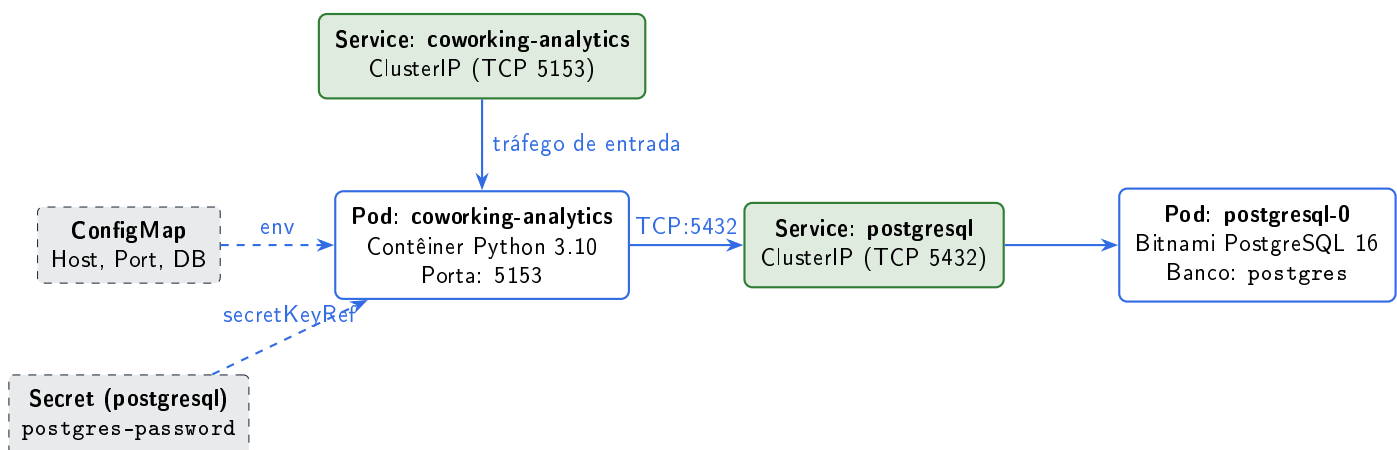
2.1 Pipeline de Integração e Entrega Contínua (CI/CD)

O fluxo ilustra o caminho do código desde o commit até a execução em produção:



2.2 Topologia Interna no Cluster Kubernetes

Representação da comunicação entre os pods no namespace `default`:



3 Detalhamento dos Componentes & Tecnologias

3.1 Componentes Locais & Ferramentas de Linha de Comando

- **AWS CLI v2:** Interface unificada para interação com APIs da Amazon Web Services. Utilizada para verificação de permissões do IAM com `aws sts get-caller-identity` e gerenciamento do ciclo de vida de imagens no ECR.

- **eksctl**: Utilitário CLI oficial para criação e gestão de clusters EKS na AWS. Encapsula pilhas do CloudFormation para provisionar automaticamente instâncias EC2, nós gerenciados, sub-redes e grupos de segurança.
- **kubectl**: Cliente padrão de controle do Kubernetes. Responsável por traduzir comandos de administração em requisições REST direcionadas ao *kube-apiserver*.
- **Helm 3**: Gerenciador de pacotes para Kubernetes. Utiliza *Charts* parametrizados para instalar softwares complexos como bancos de dados, cuidando de StatefulSets, volumes persistentes e Secrets em um comando.

3.2 Recursos e Serviços da Nuvem AWS

- **AWS EKS (Elastic Kubernetes Service)**: Plataforma de orquestração gerenciada compatível com Kubernetes de alta disponibilidade, eliminando a sobrecarga de operar o *Control Plane* e etcd.
- **AWS ECR (Elastic Container Registry)**: Registro de contêineres Docker gerenciado, integrado nativamente às políticas de IAM da AWS para autorização de *push/pull*.
- **AWS CodeBuild**: Serviço de integração contínua totalmente gerenciado e serverless. Compila o código-fonte, roda testes de integridade, constrói imagens Docker em modo privilegiado (*Privileged Mode*) e envia os artefatos finais ao ECR.
- **AWS CloudWatch**: Repositório centralizado de observabilidade e monitoramento, coletando registros de compilação (*build logs*) e métricas operacionais dos contêineres.

4 Análise de Falhas e Soluções (Troubleshooting Log)

Caso 1: Conflito de Versão Flask vs. Werkzeug

Sintoma: O pod entrava em `CrashLoopBackOff` acusando o erro:

```
ImportError: cannot import name 'url_quote' from 'werkzeug.urls'
```

Causa Raiz: O `Flask==2.2.2` utiliza a função utilitária `url_quote`, que foi descontinuada e removida na versão `Werkzeug 3.0+`. O arquivo `requirements.txt` não possuía fixação de versão para a biblioteca `Werkzeug`.

Solução: Fixação explícita da biblioteca no `requirements.txt` para `Werkzeug==2.2.2`, permitindo o build consistente e estável do contêiner.

Caso 2: Bloqueio no Rollout pelo ReadinessProbe

Sintoma: O novo pod permanecia em `0/1 Running` indefinidamente, impedindo a conclusão do comando `kubectl rollout status`.

Causa Raiz: O endpoint `/readiness_check` invocava internamente uma classe de modelo `Token` que não estava declarada ou importada no escopo do arquivo `app.py`, resultando em erro 500 no probe de prontidão.

Solução: No manifesto `deployment.yaml`, o `readinessProbe` foi ajustado para testar o endpoint funcional `/health_check`, garantindo a transição segura para `1/1 Ready`.

5 Respostas às Perguntas de Destaque (Standout Suggestions)

1. **Alocação Razoável de Memória e CPU no Kubernetes:** No manifesto `deployment.yaml`, foram estipulados limites precisos:
 - *Requests:* `cpu: 100m, memory: 64Mi` – Garante que o agendador (*scheduler*) do Kubernetes aloque o pod apenas em nós com recursos suficientes disponíveis.
 - *Limits:* `cpu: 250m, memory: 256Mi` – Cria uma barreira de proteção contra consumo excessivo de memória (*Memory Leaks*), prevenindo indisponibilidade geral do nó por exaustão.
2. **Melhor Tipo de Instância EC2 para a Aplicação:** A família de instâncias recomendada é a `t4g.small` (ou `t3.small`). Baseadas no processador proprietário **AWS Graviton2** (arquitetura ARM64), as instâncias `t4g` entregam até 40% a mais de performance com custo 20% menor que a família x86 tradicional, atendendo perfeitamente a microsserviços web com perfil de tráfego intermitente.
3. **Estratégias de Otimização e Redução de Custos:**
 - **Instâncias Spot & Karpenter:** Utilização de instâncias Spot em grupos de nós secundários para cargas tolerantes a interrupções, viabilizando economia de até 90% em relação ao preço sob demanda.
 - **Autoescalabilidade (HPA & Cluster Autoscaler):** Desligamento dinâmico de pods e redução de nós em períodos ociosos ou fora do horário comercial.